

Algorithm for Bioinformatics

Exercise 9 David Riemer, Usama Mehmood

Problem 1

We are given text $T[0..n]$, pattern $P[0..m]$, and integer k . We need to return all indices i such that there exists some length m' with

$$\text{dist}_{\text{edit}}(T[i..i + m'], P[0..m]) \leq k.$$

If edit distance $\leq k$, then the alignment consists of at most k edit operations and at most $k + 1$ exact matching blocks between edits.

LCE lets us jump over each exact block in $O(1)$ time.

~>

After linear preprocessing (time & space), we can find LCEs in $O(1)$ time.

However we need to decide which edit to use at each mismatch:

$$\text{LCE}(P, a, T, b) = \max\{\ell \mid P[a..a + \ell] = T[b..b + \ell]\}.$$

We define a state (a, b) as follows:

a : how many characters of P have been consumed (next to match is $P[a]$)

b : current position in T (next to match is $T[b]$)

We always "compress" states by immediately applying LCE so that (a, b) is at a mismatch boundary or end.

If we draw a DP table with pattern index a on one axis and text index b on the other, then lines of constant $(b - a)$ are diagonals.

d captures the net shift caused by insertions/deletions.

Substitution advances both a and $b \rightarrow d$ unchanged.

Delete from pattern ($a+ = 1$, b same) $\rightarrow d = b - a$ decreases by 1.

Delete from text ($b+ = 1$, a same) $\rightarrow d$ increases by 1.

So after at most e edits, d can only deviate from its initial value by at most e .

Edit	Effect on (a)	Effect on (b)	Effect on (d=b-a)
Substitute	+1	+1	unchanged
Delete from pattern	+1	+0	-1
Delete from text	+0	+1	+1

For each start index i , we test whether P can match starting at $T[i]$ with $\leq k$ edits.

We build a data structure for $O(1)$ LCE queries on P vs T .

We maintain an array $best[d]$ meaning, the maximum pattern index a we can reach (after LCE compression).

If $best[d] = a$, then the corresponding text index is always $b = a + d$.

Initialization (0 edits)

1. Start at $(a, b) = (0, i)$
2. Jump by LCE: $\ell = LCE(P, 0, T, i)$, $a = \ell$, $b = i + \ell$
3. Set $d = b - a$, store $best_0[d] = a$.

If $a = m$, we already matched.

Transition (from edit layer e to $e + 1$)

For each diagonal d present in $best$:

1. recover (a, b) as $a = best[d]$, $b = a + d$
2. if $a = m$: success

Otherwise we generate up to 3 next states (one edit), then LCE-jump:

Substitute

$$(a', b') = (a + 1, b + 1)$$

Delete from pattern

$$(a', b') = (a + 1, b)$$

Delete from text

$$(a', b') = (a, b + 1)$$

Then compress each successor:

$$\ell = LCE(P, a', T, b'), \quad (a'', b'') = (a' + \ell, b' + \ell)$$

We compute $d'' = b'' - a''$ and update:

$$best_{next}[d''] = \max(best_{next}[d''], a'')$$

After k layers, if no state reaches $a = m$, then i is not a match.

Example

The instance of the example was created by GPT 5.2, however the walkthrough is correct.

Initialization: ($e = 0$)

Start at $(a, b) = (0, 0)$.

- $LCE(P, 0, T, 0) = 1$
- New state: $(a, b) = (1, 1)$
- $d = b - a = 0$

$$best_0 : d = 0 \rightarrow a = 1$$

Mismatch: $P[1] = b, T[1] = x$

Edit layer $e = 1$

Process diagonal $d = 0, a = 1, b = 1$.

1. Substitute

$$\begin{aligned}
 (a', b') &= (2, 2) \\
 LCE(P, 2, T, 2) &= 1 \text{ ("c" matches)} \\
 &\rightarrow (a'', b'') = (3, 3)
 \end{aligned}$$

Pattern consumed: return.

2. Delete from pattern

$$\begin{aligned}
 (a', b') &= (2, 1) \\
 LCE(P, 2, T, 1) &= 0 \\
 &\rightarrow (2, 1)
 \end{aligned}$$

3. Delete from text

$$\begin{aligned}
 (a', b') &= (1, 2) \\
 LCE(P, 1, T, 2) &= 2 \text{ ("bc" matches)} \\
 &\rightarrow (3, 4)
 \end{aligned}$$

Pattern consumed: return.

Found a match with ≤ 1 edit.